

Recursion And Memoization Logic

The key to **3148. Maximum Difference Score in a Grid** is understanding what the recursion is actually returning.

The biggest mistake would be to think:

```
dp[i][j] = maximum score starting from (i,j)
```

That sounds reasonable, but it makes the recurrence awkward because the move can jump to **any** cell below/right.

A much cleaner way is to define the DP as:

```
dp[i][j] = maximum total score of a valid path whose first cell is (i,j) and which makes at least one move.
```

1. First understand the score

Suppose:

```
text
grid =

9  5  7
8  9  6
6  7 14
```

If we go:

```
text
5 → 7 → 14
```

score:

```
text
(7 - 5) + (14 - 7)
= 2 + 7
= 9
```

Notice something important:

```
text
(7 - 5) + (14 - 7)
= 14 - 5
```

The middle values **cancel**.

So for any sequence:

```
text
A → B → C → D
```

the total score is:

```
text
(B-A) + (C-B) + (D-C)
= D-A
```

Therefore, the entire problem is actually:

```
Find two cells (i,j) and (x,y) such that (x,y) is strictly below/right of (i,j), and maximize:
grid[x][y] - grid[i][j]
```

The intermediate moves don't matter for the final score.

2. Recursion idea

Suppose we are currently at:

```
text
(i,j)
```

We can jump to **any** cell:

```
text
(i+1, j)
(i+2, j)
...
(i, j+1)
(i, j+2)
...
(i+2, j+5)
...
```

So from (i, j) , we have two kinds of choices:

Choice 1: Make a move directly

Go to some valid cell (x, y) .

Score:

```
C++
grid[x][y] - grid[i][j]
```

Choice 2: Move to a cell and continue from there

For example:

```
text
(i, j) → (x, y) → another cell
```

The total becomes:

```
text
grid[x][y] - grid[i][j] + dp[x][y]
```

But because of the cancellation property:

```
text
grid[x][y] - grid[i][j] + dp[x][y]
```

is effectively:

```
text
finalValue - grid[i][j]
```

3. Recurrence

Define:

```
C++
dp[i][j] = maximum score obtainable starting from (i, j)
```

Then:

```
text
dp[i][j]
=
max(
    grid[x][y] - grid[i][j] + dp[x][y]
)
```

for every valid (x, y) below/right.

But there is a subtle issue.

If (x, y) is the **last cell**, then $dp[x][y]$ cannot be used because the problem requires at least one move and (x, y) makes no move.

So conceptually:

```
text
dp[i][j] =
max over every reachable (x, y):

    grid[x][y] - grid[i][j]

OR

    grid[x][y] - grid[i][j] + dp[x][y]
```

4. But we can simplify the recurrence dramatically

Because:

```
text
grid[x][y] - grid[i][j] + dp[x][y]
```

and

```
text
dp[x][y]
```

represents the best continuation.

Actually, if we define:

```
text
best[i][j] = maximum value reachable from (i,j)
```

then the answer from (i,j) is simply:

```
text
best[i][j] - grid[i][j]
```

And:

```
text
best[i][j] =
max(
    grid[x][y]
)
```

over all cells below/right.

That gives the cleanest solution.

5. Memoization recursion

We can write:

```
C++
class Solution {
public:
    int solve(int i, int j, vector<vector<int>>& grid, vector<vector<int>>& dp) {
        int m = grid.size();
        int n = grid[0].size();

        if (dp[i][j] != -1)
            return dp[i][j];

        int best = -1;

        // Move down to any row
        for (int x = i + 1; x < m; x++) {
            best = max(best, grid[x][j]);
        }

        // Move right to any column
        for (int y = j + 1; y < n; y++) {
            best = max(best, grid[i][y]);
        }

        // Move diagonally down-right
        for (int x = i + 1; x < m; x++) {
            for (int y = j + 1; y < n; y++) {
                best = max(best, grid[x][y]);
            }
        }

        return dp[i][j] = best;
    }

    int maxScore(vector<vector<int>>& grid) {
        int m = grid.size();
        int n = grid[0].size();

        vector<vector<int>> dp(m, vector<int>(n, -1));
```

```

int ans = INT_MIN;

for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        if (i == m - 1 && j == n - 1)
            continue;

        int mx = solve(i, j, grid, dp);

        ans = max(ans, mx - grid[i][j]);
    }
}

return ans;
}
};

```

However, **don't use this code** for the actual constraints.

Why?

Because every `solve(i,j)` scans potentially $O(m*n)$ cells, resulting in roughly:

```

text
O((m*n)^2)

```

which is too slow for 10^5 cells.

The recursion concept is useful for understanding, but we need to optimize it.

6. The important observation for optimized DP

Look at:

```

text
dp[i][j] = maximum value anywhere below/right of (i,j)

```

We don't need to search all those cells repeatedly.

For example:

```

text
9  5  7  3
8  9  6  1
6  7 14  3
2  5  3  1

```

For (0,1):

```

text
5

```

Everything reachable is:

```

text
9  6  1
7 14  3
5  3  1

```

Maximum:

```

text
14

```

Therefore:

```

text
answer from (0,1) = 14 - 5 = 9

```

7. Recurrence using suffix maximum

Define:

```

text
maxReach[i][j]

```

as:

Maximum grid value reachable from (i, j) by moving down/right.

Then:

```
text
maxReach[i][j]
=
max(
    grid[i+1][j],
    grid[i][j+1],
    maxReach[i+1][j],
    maxReach[i][j+1]
)
```

This is the important recurrence.

Why?

From (i, j) , anything reachable belongs to either:

```
text
↓
(i+1, j)
```

or:

```
text
→
(i, j+1)
```

Once we reach either of those cells, their `maxReach` contains everything reachable from there.

Therefore:

```
text
maxReach[i][j]
=
max(
    immediate down cell,
    immediate right cell,
    best reachable from down,
    best reachable from right
)
```

8. Where memoization comes in

Suppose we calculate:

```
text
solve(0, 0)
```

It asks:

```
text
solve(1, 0)
solve(0, 1)
```

Then:

```
text
solve(1, 0)
```

asks:

```
text
solve(2, 0)
solve(1, 1)
```

And:

```
text
solve(0, 1)
```

also eventually asks:

```
text
solve(1, 1)
```

So without memoization:

```
text
solve(1,1)
```

would be calculated multiple times.

With:

```
C++
if (dp[i][j] != -1)
    return dp[i][j];
```

we calculate it once.

This changes the recursion from exponential-like repeated work to essentially:

```
text
O(m*n)
```

states.

9. But there is an even cleaner DP

We actually don't need `maxReach` to calculate the final answer.

Process the matrix from **bottom-right toward top-left**.

Maintain:

```
text
mx = maximum value seen in the region below/right
```

But we need to be careful because only cells strictly below/right are valid.

A standard optimized solution uses a suffix maximum structure.

For every cell:

```
text
best = maximum grid value in rectangle
      [i...m-1][j...n-1]
```

Then:

```
text
answer = max(answer, bestBelowRight - grid[i][j])
```

10. The most important conceptual point

You asked:

```
"how here recurrence relation will be the answer?"
```

The answer is:

Recurrence doesn't directly equal the final answer.

The recurrence calculates:

```
text
dp[i][j]
```

which represents:

```
the best destination/value obtainable from (i,j) .
```

Then we calculate:

```
text
dp[i][j] - grid[i][j]
```

which represents:

```
the maximum score if we start at (i,j) .
```

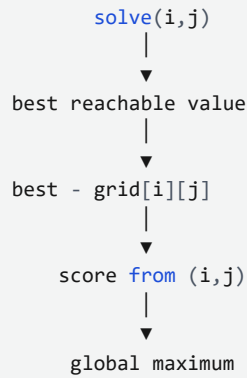
Finally:

```
text
ans = max(ans, dp[i][j] - grid[i][j])
```

because we are allowed to **start at any cell**.

So the flow is:

text



One more important correction

There is an even simpler way to think about the original problem:

text

$(A \rightarrow B) + (B \rightarrow C) + (C \rightarrow D)$

becomes:

text

$D - A$

So you don't actually need recursion at all to solve the problem.

The problem is fundamentally:

text

maximum value in a reachable lower-right region
-
current value

The recursion/memoization formulation is useful for learning DP, but for the actual $m \cdot n \leq 10^5$ constraint, the optimized suffix-max DP is the approach you should implement.

If you want to learn this as **pure recursion** → **memoization** → **bottom-up DP** → **$O(mn)$ optimized code**, that progression is actually the best way to understand this problem.

Yes. I'll give you the progression from **brute-force recursion** → **memoization** → **bottom-up DP** → **optimized $O(mn)$** so you can see exactly how the recurrence evolves.

One correction to my previous answer: the brute-force recursion I showed was unnecessarily complicated. Because the score telescopes, we can formulate the recursion much more cleanly.

1. Pure recursion – for understanding

Here:

C++

```
solve(i, j)
```

means:

Maximum score obtainable if we start at (i, j) and make at least one move.

C++

```
class Solution {
public:
    int solve(int i, int j, vector<vector<int>>& grid) {
        int m = grid.size();
        int n = grid[0].size();

        int ans = INT_MIN;

        // Move to any cell below
        for (int x = i + 1; x < m; x++) {
            ans = max(ans, grid[x][j] - grid[i][j]);

            // Continue from (x, j)
            ans = max(ans, grid[x][j] - grid[i][j] + solve(x, j, grid));
        }
    }
};
```

```

// Move to any cell on the right
for (int y = j + 1; y < n; y++) {
    ans = max(ans, grid[i][y] - grid[i][j]);

    // Continue from (i,y)
    ans = max(ans, grid[i][y] - grid[i][j] + solve(i, y, grid));
}

// Move to any lower-right cell
for (int x = i + 1; x < m; x++) {
    for (int y = j + 1; y < n; y++) {
        ans = max(ans, grid[x][y] - grid[i][j]);
        ans = max(ans, grid[x][y] - grid[i][j] + solve(x, y, grid));
    }
}

return ans;
}

int maxScore(vector<vector<int>>& grid) {
    int m = grid.size();
    int n = grid[0].size();

    int ans = INT_MIN;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            ans = max(ans, solve(i, j, grid));
        }
    }

    return ans;
}
};

```

Problem

This is **far too slow**.

The same states are calculated again and again.

2. Recursion + Memoization

Now cache:

```

C++
dp[i][j]

```

where:

```

text
dp[i][j] = best score starting from (i,j)

```

```

C++
class Solution {
public:
    int solve(int i, int j, vector<vector<int>>& grid, vector<vector<int>>& dp) {
        if (dp[i][j] != INT_MIN)
            return dp[i][j];

        int m = grid.size();
        int n = grid[0].size();

        int ans = INT_MIN;

        // Go down
        for (int x = i + 1; x < m; x++) {
            ans = max(ans, grid[x][j] - grid[i][j]);

            int next = solve(x, j, grid, dp);
            ans = max(ans, grid[x][j] - grid[i][j] + next);
        }

        // Go right

```

```

    for (int y = j + 1; y < n; y++) {
        ans = max(ans, grid[i][y] - grid[i][j]);

        int next = solve(i, y, grid, dp);
        ans = max(ans, grid[i][y] - grid[i][j] + next);
    }

    // Go diagonally / Lower-right
    for (int x = i + 1; x < m; x++) {
        for (int y = j + 1; y < n; y++) {
            ans = max(ans, grid[x][y] - grid[i][j]);

            int next = solve(x, y, grid, dp);
            ans = max(ans, grid[x][y] - grid[i][j] + next);
        }
    }

    return dp[i][j] = ans;
}

int maxScore(vector<vector<int>>& grid) {
    int m = grid.size();
    int n = grid[0].size();

    vector<vector<int>> dp(m, vector<int>(n, INT_MIN));

    int ans = INT_MIN;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            ans = max(ans, solve(i, j, grid, dp));
        }
    }

    return ans;
}
};

```

This removes repeated recursion, but there is still a major problem:

```

C++
for (x ...)
for (y ...)

```

Every state scans a huge number of cells.

So this still isn't good enough for:

```

text
m * n <= 100000

```

3. Better recurrence

Here's where the important observation comes in.

For:

```

text
A → B → C → D

```

score is:

```

text
(B-A) + (C-B) + (D-C)

```

Everything cancels:

```

text
= D-A

```

Therefore:

```

text
maximum score starting at (i,j)
=
maximum reachable value - grid[i][j]

```

So define:

```
text
dp[i][j] = maximum value reachable from (i,j)
```

Then:

```
text
dp[i][j] =
max(
    grid[i+1][j],
    grid[i][j+1],
    dp[i+1][j],
    dp[i][j+1]
)
```

4. Memoization using this recurrence

This is the much cleaner recursive DP version:

```
C++
class Solution {
public:
    int solve(int i, int j, vector<vector<int>>& grid, vector<vector<int>>& dp) {
        int m = grid.size();
        int n = grid[0].size();

        if (dp[i][j] != -1)
            return dp[i][j];

        int best = INT_MIN;

        // Go down
        if (i + 1 < m) {
            best = max(best, grid[i + 1][j]);
            best = max(best, solve(i + 1, j, grid, dp));
        }

        // Go right
        if (j + 1 < n) {
            best = max(best, grid[i][j + 1]);
            best = max(best, solve(i, j + 1, grid, dp));
        }

        return dp[i][j] = best;
    }

    int maxScore(vector<vector<int>>& grid) {
        int m = grid.size();
        int n = grid[0].size();

        vector<vector<int>> dp(m, vector<int>(n, -1));

        int ans = INT_MIN;

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (i == m - 1 && j == n - 1)
                    continue;

                int best = solve(i, j, grid, dp);

                ans = max(ans, best - grid[i][j]);
            }
        }

        return ans;
    }
};
```

Complexity

There are:

text

m × n

states.

Each state does only constant work.

Therefore:

text

Time = $O(m \times n)$
Space = $O(m \times n)$

This is already acceptable because:

text

m × n <= 100000

5. Bottom-Up DP

Now remove recursion.

Since $dp[i][j]$ depends on:

text

$dp[i+1][j]$
 $dp[i][j+1]$

we process:

text

bottom → top
right → left

Code:

C++

```
class Solution {
public:
    int maxScore(vector<vector<int>>& grid) {
        int m = grid.size();
        int n = grid[0].size();

        vector<vector<int>> dp(m, vector<int>(n, INT_MIN));

        int ans = INT_MIN;

        for (int i = m - 1; i >= 0; i--) {
            for (int j = n - 1; j >= 0; j--) {

                // Down
                if (i + 1 < m) {
                    dp[i][j] = max(dp[i][j], grid[i + 1][j]);
                    dp[i][j] = max(dp[i][j], dp[i + 1][j]);
                }

                // Right
                if (j + 1 < n) {
                    dp[i][j] = max(dp[i][j], grid[i][j + 1]);
                    dp[i][j] = max(dp[i][j], dp[i][j + 1]);
                }

                // If there is at least one valid move
                if (dp[i][j] != INT_MIN) {
                    ans = max(ans, dp[i][j] - grid[i][j]);
                }
            }
        }

        return ans;
    }
};
```

This is:

text

Time: $O(m \times n)$
Space: $O(m \times n)$

6. $O(n)$ Space Optimization

We don't actually need the entire `dp` matrix.

For each cell, we only need:

text

```
dp[i+1][j]
dp[i][j+1]
```

So we can use a 1D array.

C++

```
class Solution {
public:
    int maxScore(vector<vector<int>>& grid) {
        int m = grid.size();
        int n = grid[0].size();

        vector<int> dp(n, INT_MIN);

        int ans = INT_MIN;

        for (int i = m - 1; i >= 0; i--) {
            for (int j = n - 1; j >= 0; j--) {

                int best = INT_MIN;

                // Down
                if (i + 1 < m) {
                    best = max(best, grid[i + 1][j]);
                    best = max(best, dp[j]);
                }

                // Right
                if (j + 1 < n) {
                    best = max(best, grid[i][j + 1]);
                    best = max(best, dp[j + 1]);
                }

                dp[j] = best;

                if (best != INT_MIN) {
                    ans = max(ans, best - grid[i][j]);
                }
            }
        }

        return ans;
    }
};
```

Complexity:

text

Time = $O(m \times n)$
Space = $O(n)$

Which one should you learn?

For understanding DP, follow this progression:

text

```
Problem
↓
Score telescopes: final - initial
↓
dp[i][j] = best reachable value
↓
Recursion
↓
Memoization
↓
Bottom-Up DP
↓
Space Optimization
```

The recurrence you should remember

C++

```
dp[i][j] = max(dp[i+1][j], dp[i][j+1],
               grid[i+1][j], grid[i][j+1]);
```

And the final answer is:

C++

```
ans = max(ans, dp[i][j] - grid[i][j]);
```

For LeetCode submission, use the 1D-space version. For learning the recurrence, use the memoization version first.

Absolutely. This is the right code to dry-run because once you see the `dp` table being filled, the recurrence becomes much clearer.

Given:

text

```
grid =
```

```
9  5  7  3
8  9  6  1
6  7 14  3
2  5  3  1
```

We use:

C++

```
dp[i][j] = maximum value we can reach from (i,j)
```

by making at least one move **down or right**.

1. Initial DP table

Initially:

text

```
dp =
```

```
-∞  -∞  -∞  -∞
-∞  -∞  -∞  -∞
-∞  -∞  -∞  -∞
-∞  -∞  -∞  -∞
```

`INT_MIN` is represented as `-∞` here.

We process:

C++

```
for (int i = m - 1; i >= 0; i--)
```

so:

text

```
i = 3
i = 2
i = 1
i = 0
```

And for each row:

C++

```
j = n - 1 → 0
```

so we're moving:

text

bottom-right → top-left

2. Start at (3,3)

Grid:

text

(3,3) = 1

There is:

text

down? NO
right? NO

Therefore:

text

dp[3][3] = INT_MIN

DP:

text

-∞	-∞	-∞	-∞
-∞	-∞	-∞	-∞
-∞	-∞	-∞	-∞
-∞	-∞	-∞	-∞

No valid move exists from (3,3).

3. (3,2)

text

grid[3][2] = 3

There is no down.

But there is right:

text

(3,2) → (3,3)

The code:

C++

```
dp[i][j] = max(dp[i][j], grid[i][j + 1]);
```

means:

text

```
dp[3][2] = max(-∞, grid[3][3])  
          = max(-∞, 1)  
          = 1
```

Then:

C++

```
dp[i][j] = max(dp[i][j], dp[i][j + 1]);
```

becomes:

text

```
dp[3][2] = max(1, dp[3][3])  
          = max(1, -∞)  
          = 1
```

So:

text

dp[3][2] = 1

And score:

```
text
1 - grid[3][2]
= 1 - 3
= -2
```

DP:

```
text
-∞ -∞ -∞ -∞
-∞ -∞ -∞ -∞
-∞ -∞ -∞ -∞
-∞ -∞ 1 -∞
```

4. (3,1)

```
text
grid[3][1] = 5
```

Right cell:

```
text
grid[3][2] = 3
```

So:

```
text
dp[3][1] = max(-∞, 3)
          = 3
```

Then:

```
text
dp[3][1] = max(3, dp[3][2])
          = max(3, 1)
          = 3
```

Therefore:

```
text
dp[3][1] = 3
```

Score:

```
text
3 - 5 = -2
```

DP:

```
text
-∞ -∞ -∞ -∞
-∞ -∞ -∞ -∞
-∞ -∞ -∞ -∞
-∞ 3 1 -∞
```

5. (3,0)

```
text
grid[3][0] = 2
```

Right:

```
text
grid[3][1] = 5
```

Therefore:

```
text
dp[3][0] = max(-∞, 5)
          = 5
```

Then:

text

```
dp[3][0] = max(5, dp[3][1])
           = max(5, 3)
           = 5
```

So:

text

```
dp[3][0] = 5
```

Score:

text

```
5 - 2 = 3
```

DP:

text

```
-∞ -∞ -∞ -∞
-∞ -∞ -∞ -∞
-∞ -∞ -∞ -∞
5  3  1 -∞
```

Current:

text

```
ans = 3
```

6. Row 2

Now:

text

```
i = 2
```

Grid row:

text

```
6  7  14  3
```

(2, 3)

Value:

text

```
grid[2][3] = 3
```

Down:

text

```
grid[3][3] = 1
```

So:

text

```
dp[2][3] = max(-∞, 1)
           = 1
```

Then:

text

```
dp[2][3] = max(1, dp[3][3])
           = max(1, -∞)
           = 1
```

Thus:

text

```
dp[2][3] = 1
```

Score:

text

```
1 - 3 = -2
```

DP:

```
text
-∞ -∞ -∞ -∞
-∞ -∞ -∞ -∞
-∞ -∞ -∞ 1
5 3 1 -∞
```

7. (2,2)

This is the important one.

```
text
grid[2][2] = 14
```

Down:

```
text
grid[3][2] = 3
```

and:

```
text
dp[3][2] = 1
```

So down gives:

```
text
max(3, 1) = 3
```

Right:

```
text
grid[2][3] = 3
```

and:

```
text
dp[2][3] = 1
```

So right gives:

```
text
max(3, 1) = 3
```

Therefore:

```
text
dp[2][2] = 3
```

Score:

```
text
3 - 14 = -11
```

DP:

```
text
-∞ -∞ -∞ -∞
-∞ -∞ -∞ -∞
-∞ -∞ 3 1
5 3 1 -∞
```

Why is `dp[2][2] = 3`?

Because from:

```
text
14
```

you can only go to:

```
text
3
```

or eventually reach:

```
text
1
```

The maximum reachable value is 3.

8. (2,1)

text

```
grid[2][1] = 7
```

Down:

text

```
grid[3][1] = 5
dp[3][1] = 3
```

So:

text

```
max(5, 3) = 5
```

Right:

text

```
grid[2][2] = 14
dp[2][2] = 3
```

So:

text

```
max(14, 3) = 14
```

Therefore:

text

```
dp[2][1] = max(5, 3, 14, 3)
          = 14
```

Score:

text

```
14 - 7 = 7
```

Now:

text

```
ans = max(3, 7)
     = 7
```

DP:

text

```
-∞  -∞  -∞  -∞
-∞  -∞  -∞  -∞
-∞  14  3   1
5   3   1  -∞
```

9. (2,0)

text

```
grid[2][0] = 6
```

Down:

text

```
grid[3][0] = 2
dp[3][0] = 5
```

So:

text

```
max(2, 5) = 5
```

Right:

text

```
grid[2][1] = 7
dp[2][1] = 14
```

So:

```
text
max(7,14) = 14
```

Therefore:

```
text
dp[2][0] = 14
```

Score:

```
text
14 - 6 = 8
```

So:

```
text
ans = 8
```

DP:

```
text
-∞ -∞ -∞ -∞
-∞ -∞ -∞ -∞
14 14 3 1
5 3 1 -∞
```

10. Row 1

Now:

```
text
i = 1
```

(1,3)

```
text
grid[1][3] = 1
```

Down:

```
text
grid[2][3] = 3
dp[2][3] = 1
```

Therefore:

```
text
dp[1][3] = max(3,1)
           = 3
```

Score:

```
text
3 - 1 = 2
```

So:

```
text
ans = 8
```

11. (1,2)

```
text
grid[1][2] = 6
```

Down:

```
text
grid[2][2] = 14
dp[2][2] = 3
```

Therefore:

```
text
max(14,3) = 14
```

Right:

```
text
grid[1][3] = 1
dp[1][3] = 3
```

Therefore:

```
text
max(1,3) = 3
```

So:

```
text
dp[1][2] = max(14,3,1,3)
          = 14
```

Score:

```
text
14 - 6 = 8
```

ans remains:

```
text
8
```

DP:

```
text
-∞  -∞  -∞  -∞
-∞  -∞  14  3
14  14  3  1
5   3  1  -∞
```

12. (1,1)

```
text
grid[1][1] = 9
```

Down:

```
text
grid[2][1] = 7
dp[2][1] = 14
```

So:

```
text
max(7,14) = 14
```

Right:

```
text
grid[1][2] = 6
dp[1][2] = 14
```

So:

```
text
max(6,14) = 14
```

Therefore:

```
text
dp[1][1] = 14
```

Score:

```
text
14 - 9 = 5
```

DP:

```
text
-∞ -∞ -∞ -∞
-∞ 14 14 3
14 14 3 1
5 3 1 -∞
```

13. (1,0)

```
text
grid[1][0] = 8
```

Down:

```
text
grid[2][0] = 6
dp[2][0] = 14
```

So:

```
text
max(6,14) = 14
```

Right:

```
text
grid[1][1] = 9
dp[1][1] = 14
```

So:

```
text
max(9,14) = 14
```

Therefore:

```
text
dp[1][0] = 14
```

Score:

```
text
14 - 8 = 6
```

14. Row 0

Now:

```
text
i = 0
```

(0,3)

```
text
grid[0][3] = 3
```

Down:

```
text
grid[1][3] = 1
dp[1][3] = 3
```

Therefore:

```
text
dp[0][3] = max(1,3)
          = 3
```

Score:

```
text
3 - 3 = 0
```

15. (0,2)

text

```
grid[0][2] = 7
```

Down:

text

```
grid[1][2] = 6  
dp[1][2] = 14
```

So:

text

```
max(6,14) = 14
```

Right:

text

```
grid[0][3] = 3  
dp[0][3] = 3
```

So:

text

```
max(3,3) = 3
```

Therefore:

text

```
dp[0][2] = 14
```

Score:

text

```
14 - 7 = 7
```

16. (0,1) — the important cell

text

```
grid[0][1] = 5
```

Down:

text

```
grid[1][1] = 9  
dp[1][1] = 14
```

So:

text

```
max(9,14) = 14
```

Right:

text

```
grid[0][2] = 7  
dp[0][2] = 14
```

So:

text

```
max(7,14) = 14
```

Therefore:

text

```
dp[0][1] = 14
```

And:

```
text
score = dp[0][1] - grid[0][1]

      = 14 - 5

      = 9
```

So:

```
text
ans = 9
```

This corresponds exactly to the optimal path:

```
text
5 → 7 → 14
```

Score:

```
text
(7 - 5) + (14 - 7)
= 2 + 7
= 9
```

17. (0,0)

Finally:

```
text
grid[0][0] = 9
```

Down:

```
text
grid[1][0] = 8
dp[1][0] = 14
```

So:

```
text
max(8, 14) = 14
```

Right:

```
text
grid[0][1] = 5
dp[0][1] = 14
```

So:

```
text
max(5, 14) = 14
```

Therefore:

```
text
dp[0][0] = 14
```

Score:

```
text
14 - 9 = 5
```

ans remains:

```
text
9
```

Final DP table

After the entire loop:

```
text
dp =

14  14  14  3
14  14  14  3
14  14  3  1
5   3  1  -∞
```

And:

```
text
ans = 9
```

Why does this DP table look like this?

Take:

```
text
dp[0][1] = 14
```

because starting at:

```
text
5
```

you can eventually reach:

```
text
14
```

through:

```
text
5 → 7 → 14
```

Take:

```
text
dp[0][0] = 14
```

because starting at 9 , you can eventually reach 14 :

```
text
9 → 9 → 14
```

Take:

```
text
dp[2][2] = 3
```

because starting at 14 , the only possible destinations are:

```
text
14 → 3
```

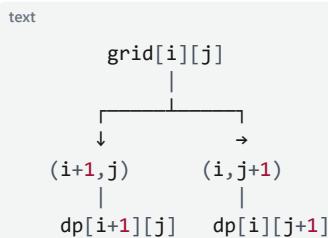
or:

```
text
14 → 3 → 1
```

so the maximum reachable value is only 3 .

The recurrence in one picture

For a cell:



Therefore:

C++

```
dp[i][j] = max(
    grid[i+1][j],
    dp[i+1][j],
    grid[i][j+1],
    dp[i][j+1]
);
```

And then:

C++

```
ans = max(ans, dp[i][j] - grid[i][j]);
```

The most important thing to understand

`dp[i][j]` does not store the score.

It stores:

the maximum value that can be reached from (i, j)

Then we convert that into a score:

text

maximum reachable value - current value

That's why:

text

`dp[0][1] = 14`

but the answer contributed by $(0, 1)$ is:

text

`14 - 5 = 9`

That distinction is the core of this DP.

Yes. This is the part that usually feels confusing: how can a 1D `dp` array replace the 2D `dp` table?

The answer is that while processing (i, j) , we only need two pieces of information:

- `dp[j]` → value from the rowbelow
- `dp[j+1]` → value from theright

Because we're processing from **bottom-right** → **top-left**, those are exactly the states needed.

Let's dry-run the entire code.

Given:

text

grid =

```
9  5  7  3
8  9  6  1
6  7 14  3
2  5  3  1
```

1. Initial state

There are 4 columns, so:

C++

```
vector<int> dp(n, INT_MIN);
```

Initially:

text

`dp = [-∞, -∞, -∞, -∞]`

Think of this 1D array as the current stored DP information for the rows we've already processed.

2. Why process from bottom-right?

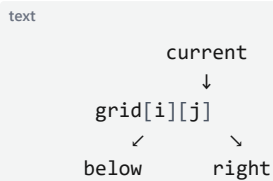
Our recurrence is:

```

text
dp[i][j] =
max(
    grid[i+1][j],
    dp[i+1][j],
    grid[i][j+1],
    dp[i][j+1]
)

```

So we need:



Therefore, we need to have already calculated:

```

text
dp[i+1][j]
dp[i][j+1]

```

That's why:

```

C++
for (int i = m - 1; i >= 0; i--)

```

and:

```

C++
for (int j = n - 1; j >= 0; j--)

```

3. Start (3,3)

Grid:

```

text
2   5   3   1
      ↑
    (3,3)

```

Current value:

```

text
grid[3][3] = 1

```

There is no down and no right.

So:

```

text
best = -∞

```

Then:

```

C++
dp[j] = best;

```

Therefore:

```

text
dp[3] = -∞

```

Array:

```

text
dp = [-∞, -∞, -∞, -∞]

```

4. (3,2)

Current:

```

text
grid[3][2] = 3

```

Down?

No.

Right?

Yes:

```
text
grid[3][3] = 1
```

So:

```
C++
best = max(-∞, 1);
```

Therefore:

```
text
best = 1
```

Then:

```
C++
best = max(best, dp[j + 1]);
```

Here:

```
text
dp[3] = -∞
```

So:

```
text
best = max(1, -∞)
      = 1
```

Now:

```
C++
dp[2] = best;
```

Therefore:

```
text
dp = [-∞, -∞, 1, -∞]
```

Notice something important:

dp[2] = 1 represents:

From (3,2), maximum reachable value is 1.

5. (3,1)

Current:

```
text
grid[3][1] = 5
```

Right:

```
text
grid[3][2] = 3
```

So:

```
text
best = 3
```

And:

```
text
dp[2] = 1
```

So:

```
text
best = max(3, 1)
      = 3
```

Update:

```
text
dp[1] = 3
```

Now:

```
text
dp = [-∞, 3, 1, -∞]
```

6. (3,0)

Current:

```
text
grid[3][0] = 2
```

Right:

```
text
grid[3][1] = 5
```

and:

```
text
dp[1] = 3
```

Therefore:

```
text
best = max(5, 3)
      = 5
```

Update:

```
text
dp[0] = 5
```

Now:

```
text
dp = [5, 3, 1, -∞]
```

So after completing row 3:

```
text
2   5   3   1
```

the 1D DP is:

```
text
dp = [5, 3, 1, -∞]
```

This is equivalent to the last row of our 2D DP:

```
text
5   3   1   -∞
```

7. Now row 2

Move to:

```
text
i = 2
```

We process:

```
text
(2,3)
(2,2)
(2,1)
(2,0)
```

(2,3)

Current:

```
text
grid[2][3] = 3
```

Down:

```
text
grid[3][3] = 1
```

And:

```
text
dp[3] = -∞
```

So:

```
text
best = max(1, -∞)
      = 1
```

Update:

```
text
dp[3] = 1
```

Now:

```
text
dp = [5, 3, 1, 1]
```

8. (2,2) — VERY IMPORTANT

Current:

```
text
grid[2][2] = 14
```

Before updating:

```
text
dp = [5, 3, 1, 1]
```

We need:

Down

```
text
grid[3][2] = 3
```

and:

```
text
dp[2] = 1
```

So:

```
text
best = max(3, 1)
      = 3
```

Right

```
text
grid[2][3] = 3
```

and:

```
text
dp[3] = 1
```

So:

text

```
best = max(3, 3, 1)
      = 3
```

Then:

C++

```
dp[2] = best;
```

So:

text

```
dp = [5, 3, 3, 1]
```

Notice what happened

Before processing (2,2):

text

```
dp[2] = 1
```

represented:

text

```
dp[3][2] = 1
```

because we're still carrying information from the row below.

After processing (2,2):

text

```
dp[2] = 3
```

now represents:

text

```
dp[2][2] = 3
```

So the same array position gets overwritten with the current row's answer.

That's the entire trick.

9. (2,1)

Current:

text

```
grid[2][1] = 7
```

Before processing:

text

```
dp = [5, 3, 3, 1]
```

Down

text

```
grid[3][1] = 5
dp[1] = 3
```

So:

text

```
max(5,3) = 5
```

Right

text

```
grid[2][2] = 14
dp[2] = 3
```

So:

text

```
max(14,3) = 14
```

Therefore:

```
text
dp[1] = 14
```

Now:

```
text
dp = [5, 14, 3, 1]
```

10. (2,0)

Current:

```
text
grid[2][0] = 6
```

Down:

```
text
grid[3][0] = 2
dp[0] = 5
```

Therefore:

```
text
max(2,5) = 5
```

Right:

```
text
grid[2][1] = 7
dp[1] = 14
```

Therefore:

```
text
max(7,14) = 14
```

So:

```
text
dp[0] = 14
```

Now:

```
text
dp = [14, 14, 3, 1]
```

11. Notice what dp represents now

We've finished row 2.

The original 2D DP would be:

```
text
14  14  3  1
```

Our 1D DP is:

```
text
[14, 14, 3, 1]
```

Exactly the same.

We just don't store the old row anymore.

12. Row 1

Now:

```
text
i = 1
```

Current dp:

```
text
[14, 14, 3, 1]
```

(1,3)

```
text
grid[1][3] = 1
```

Down:

```
text
grid[2][3] = 3
dp[3] = 1
```

So:

```
text
best = 3
```

Update:

```
text
dp[3] = 3
```

Now:

```
text
dp = [14, 14, 3, 3]
```

(1,2)

Current:

```
text
grid[1][2] = 6
```

Down:

```
text
grid[2][2] = 14
dp[2] = 3
```

So:

```
text
best = 14
```

Right:

```
text
grid[1][3] = 1
dp[3] = 3
```

So still:

```
text
best = 14
```

Update:

```
text
dp[2] = 14
```

Now:

```
text
dp = [14, 14, 14, 3]
```

(1,1)

Current:

```
text
grid[1][1] = 9
```

Down:

```
text
grid[2][1] = 7
dp[1] = 14
```

So:

```
text
best = 14
```

Right:

```
text
grid[1][2] = 6
dp[2] = 14
```

Still:

```
text
best = 14
```

Update:

```
text
dp[1] = 14
```

No visible change:

```
text
dp = [14, 14, 14, 3]
```

(1,0)

Down:

```
text
grid[2][0] = 6
dp[0] = 14
```

Right:

```
text
grid[1][1] = 9
dp[1] = 14
```

Therefore:

```
text
dp[0] = 14
```

Final:

```
text
dp = [14, 14, 14, 3]
```

13. Row 0

Now process:

```
text
i = 0
```

(0,3)

Current:

```
text
grid[0][3] = 3
```

Down:

```
text
grid[1][3] = 1
dp[3] = 3
```

Therefore:

```
text
best = 3
```

Update:

```
text
dp[3] = 3
```

(0,2)

Current:

```
text
grid[0][2] = 7
```

Down:

```
text
grid[1][2] = 6
dp[2] = 14
```

So:

```
text
best = 14
```

Right:

```
text
grid[0][3] = 3
dp[3] = 3
```

Still:

```
text
best = 14
```

Update:

```
text
dp[2] = 14
```

(0,1)

This is the important one.

Current:

```
text
grid[0][1] = 5
```

Before processing:

```
text
dp = [14, 14, 14, 3]
```

Down

```
text
grid[1][1] = 9
dp[1] = 14
```

Therefore:

```
text
best = 14
```

Right

```
text
grid[0][2] = 7
dp[2] = 14
```

Therefore:

```
text
best = 14
```

So:

```
text
dp[1] = 14
```

Now:

```
text
dp = [14, 14, 14, 3]
```

Score:

```
text
best - grid[0][1]
= 14 - 5
= 9
```

Therefore:

```
text
ans = 9
```

14. Finally (0,0)

Current:

```
text
grid[0][0] = 9
```

Down:

```
text
grid[1][0] = 8
dp[0] = 14
```

So:

```
text
best = 14
```

Right:

```
text
grid[0][1] = 5
dp[1] = 14
```

Still:

```
text
best = 14
```

Therefore:

```
text
dp[0] = 14
```

Score:

```
text
14 - 9 = 5
```

ans remains:

```
text
9
```

Final 1D DP

At the end:

```
text
dp = [14, 14, 14, 3]
```

and:

```
text
ans = 9
```

The REAL trick behind 1D DP

This is the part you need to understand.

The original 2D DP looks like:

text

```
14  14  14  3
14  14  14  3
14  14  3   1
5   3   1  -∞
```

But notice how we calculate:

text

```
dp[i][j]
```

We only need:

text

```
dp[i+1][j]    // below
dp[i][j+1]    // right
```

We don't need the entire 2D table.

So we reuse one row.

What does `dp[j]` mean?

This changes during the loop.

Before processing row `i`

text

```
dp[j]
```

means:

text

```
dp[i+1][j]
```

After processing `(i,j)`

text

```
dp[j]
```

means:

text

```
dp[i][j]
```

That's the key.

Example at `(2,2)`

Before:

text

```
dp = [5, 3, 1, 1]
      ↑
      dp[2]
```

Here:

text

```
dp[2] = dp[3][2] = 1
```

Then we calculate:

text

```
dp[2][2] = 3
```

and overwrite:

text

```
dp[2] = 3
```

So:

text

BEFORE :

```
dp[2] = dp[3][2]
      = 1
```

AFTER :

```
dp[2] = dp[2][2]
      = 3
```

This is why processing j **right** $\rightarrow$ **left** is essential.

Because when we're at (i, j) :

text

```
dp[j+1]
```

has already been updated to the current row:

text

```
dp[i][j+1]
```

while:

text

```
dp[j]
```

still contains the previous row:

text

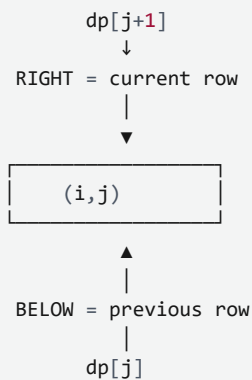
```
dp[i+1][j]
```

Perfect.

Visualize it like this

At (i, j) :

text



So:

C++

```
best = max(
    grid[i+1][j], // immediate below
    dp[j],        // best from below
    grid[i][j+1], // immediate right
    dp[j+1]       // best from right
);
```

Then:

C++

```
dp[j] = best;
```

means:

"I've finished calculating this cell. Replace the old below-row information with the current-row information."

That's **1D DP / rolling-array optimization**.

Final complexity

text

Time: $O(m \times n)$

Space: $O(n)$

The important pattern to remember for other DP problems is:

If $dp[i][j]$ only depends on the current row's $j+1$ and the previous row's j , you can often compress a 2D DP table into 1D.